MVC

& APIs REST

Plataforma de streaming

Integrantes

Brandon Mamani · Moises Palomino

REST | MVC | C# | Swagger | JSON | Versionado

Arquitectura MVC
y APIs REST

Caso 2 · Lenguaje de Programación

ASP.NET Core · C#

PREGUNTA 01

Separación de Responsabilidades en ASP.NET Core MVC

CLIENTE	ROUTING	DATOS	RESPUESTA
Navegador / App	**Controller**	**Model / DB**	**View (.cshtml)**

HTTP Request → Routing → Controller → Model → View → HTTP Response

◈ Model

Representa los datos y lógica de negocio. Incluye entidades de dominio, acceso a datos (EF Core) y validaciones. No conoce View ni Controller.

◉ View

Presenta información al usuario via Razor Pages (.cshtml). Solo recibe datos mediante ViewModels; no ejecuta lógica de negocio.

◆ Controller

Orquesta el flujo: recibe la solicitud HTTP, invoca el Model y devuelve la View correcta. Único punto de entrada desde la capa HTTP.

PREGUNTA 02

PeliculasController — Cuatro Endpoints REST en C#

GET /api/peliculas

Obtiene el listado completo. Acepta filtros opcionales.

Código: 200 OK · 204 No Content si vacío

PUT /api/peliculas/{id}

Actualiza completamente una película existente.

Código: 204 No Content · 404 Not Found · 400 Bad Request

POST /api/peliculas

Crea nueva película con datos del body JSON.

Código: 201 Created + header Location · 400 Bad Request

DELETE /api/peliculas/{id}

Elimina la película con el ID indicado.

Código: 204 No Content · 404 Not Found si no existe

💡 Los códigos de estado no son arbitrarios: 201 Created informa que se generó un recurso nuevo e incluye su ubicación; 204 No Content confirma éxito sin cuerpo de respuesta; 404 comunica semánticamente que el recurso no existe.

PREGUNTA 03

Web API vs MVC Clásico en ASP.NET Core

MVC Clásico

Devuelve: **Vistas HTML (Razor)**

Hereda de: **Controller (completo)**

Activa: **ViewBag, TempData, HTML helpers**

Ideal para: **e-commerce, portales, CMSs, SEO crítico**

Renderizado: **En servidor (SSR)**

VS

Web API

Devuelve: **JSON / XML (datos)**

Hereda de: **ControllerBase (liviano)**

Activa: **[ApiController]: validación automática + content negotiation**

Ideal para: **SPAs (React/Angular), mobile apps, microservicios**

Renderizado: **En cliente (CSR)**

En ASP.NET Core ambos coexisten en el mismo proyecto. Controllers MVC para páginas de admin + Controllers API para el frontend React/mobile, todo en una sola solución.

PREGUNTA 04

Serialización JSON: System.Text.Json vs Newtonsoft.Json

Serializar un objeto C# a JSON = convertir su estado a texto; deserializar = el proceso inverso.

System.Text.Json

Nativo .NET 6+

- ✓ Nativo, sin dependencias externas
- ✓ ~2–3× más rápido en benchmarks
- ✓ Menor consumo de memoria
- ✓ Source Generators para AOT
- ✗ Menos flexible con tipos complejos
- ✗ Sin soporte para tipos dinámicos
- ✗ Configuración más verbosa

Newtonsoft.Json

Paquete NuGet — 15+ años

- ✓ Muy maduro y estable
- ✓ Muy flexible con JsonConverters
- ✓ Soporte para tipos anónimos/dinámicos
- ✓ Ecosistema enorme (Azure SDK, etc.)
- ✗ Más lento y más memoria
- ✗ Dependencia externa
- ✗ No optimizado para AOT

Recomendación Netflix: System.Text.Json por defecto (volumen masivo); Newtonsoft solo para integraciones legacy o polimorfismo complejo.

PREGUNTA 05

Versionado de APIs — Por qué Netflix lo necesita

¿Por qué?

Netflix tiene miles de dispositivos distintos (Smart TVs 2014, iOS, Android, navegadores, consolas) consumiendo la misma API. Sin versionado, cualquier cambio obligaría a migración simultánea imposible en producción.

URL Segment

GET /api/v2/peliculas

APIs públicas — Netflix usa este

Header

X-API-Version: 2.0

APIs internas entre microservicios

Query String

?api-version=2.0

Simple, fácil de testear en browser

// Program.cs — Configuración de versionado

```
builder.Services.AddApiVersioning(opt => {
    opt.DefaultApiVersion = new ApiVersion(1, 0);
    opt.AssumeDefaultVersionWhenUnspecified = true;
    opt.ReportApiVersions = true;
    opt.ApiVersionReader = new UrlSegmentApiVersionReader();
});
// Uso: GET /api/v1/peliculas → GET /api/v2/peliculas
```

PREGUNTA 06

Documentación Automática con Swagger / OpenAPI

1. **Instalar paquete**

```
dotnet add package Swashbuckle.AspNetCore
```

2. **Configurar en Program.cs**

```
builder.Services.AddSwaggerGen(c => { c.SwaggerDoc("v1", new OpenApiInfo {...}) })
```

3. **Activar middleware**

```
app.UseSwagger(); app.UseSwaggerUI();
```

4. **Documentar con XML comments**

```
/// <summary>Obtiene todas las películas</summary>
```

● ● ● localhost:5001/swagger

GET	/api/v1/peliculas
POST	/api/v1/peliculas
DELETE	/api/v1/peliculas/{id}

Beneficios para el equipo

- **Docs siempre sincronizadas con el código**
  Nunca docs desactualizadas
- **Cliente interactivo para probar endpoints**
  Sin necesidad de Postman
- **Facilita integración de equipos**
  Frontend, QA y terceros
- **Generación automática de clientes**
  TypeScript/C# desde schema OpenAPI con NSwag

PREGUNTA 07

Diseño de API para una Plataforma de Video

ENDPOINT	MÉTODO	DESCRIPCIÓN	RESPUESTA
/api/v1/peliculas	GET	Catálogo paginado con filtros	200 + PagedResult
/api/v1/peliculas/{id}	GET	Detalle con géneros y cast	200 / 404
/api/v1/peliculas/{id}/reproducciones	POST	Inicia reproducción, registra sesión	201 + StreamToken
/api/v1/usuarios/{id}/historial	GET	Historial de visualización	200 + array
/api/v1/usuarios/{id}/lista	POST	Agrega a 'Mi Lista'	201 / 409
/api/v1/buscar?q=batman	GET	Búsqueda full-text	200 + resultados
/api/v1/peliculas/{id}	DELETE	Eliminar del catálogo (admin)	204 / 403

Convenciones REST aplicadas:

- kebab-case URLs
- GUIDs como IDs
- HTTPS obligatorio
- JWT Bearer tokens
- RFC 7807 errors
- Paginación cursor/offset

Conclusiones Individuales

Sobre el patrón MVC

El patrón MVC no es una norma rígida sino un principio de organización. En aplicaciones reales como Netflix, el modelo evoluciona hacia arquitectura de capas más granular (servicios, repositorios, dominio), donde el controlador se vuelve un adaptador delgado. La clave: cada capa con una sola razón para cambiar.

Sobre el diseño de APIs REST

Diseñar una API es diseñar un contrato. Los errores más costosos ocurren cuando ese contrato no es claro: códigos de estado incorrectos, estructuras de respuesta inconsistentes o URLs que mezclan verbos y recursos. Invertir tiempo en definirla bien y documentarla con Swagger ahorra semanas de debugging.

Sobre el versionado y la evolución

El versionado es una práctica de madurez técnica. Un sistema que nunca versiona su API acumula deuda técnica hasta que un cambio necesario rompe producción. Netflix con sus miles de clientes es el ejemplo extremo, pero incluso proyectos medianos se benefician del versionado desde el inicio.

Bibliografía

1. Microsoft. (2024). ASP.NET Core MVC Overview. Microsoft Docs. https://learn.microsoft.com/en-us/aspnet/core/mvc/overview

2. Microsoft. (2024). Create web APIs with ASP.NET Core. https://learn.microsoft.com/en-us/aspnet/core/web-api/

3. Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley. ISBN: 978-0321127426.

4. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. UC Irvine.

5. Microsoft. (2024). JSON serialization in .NET. https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview

6. dotnet/aspnet-api-versioning. (2024). ASP.NET API Versioning. https://github.com/dotnet/aspnet-api-versioning

7. Swagger. (2024). Swashbuckle.AspNetCore — Getting Started. https://github.com/domaindrivendev/Swashbuckle.AspNetCore

8. IETF. (2016). RFC 7807 — Problem Details for HTTP APIs. https://datatracker.ietf.org/doc/html/rfc7807

9. Lock, A. (2023). ASP.NET Core in Action (3rd ed.). Manning Publications. ISBN: 978-1633438200.

10. Netflix Tech Blog. (2023). Netflix's Approach to API Architecture. https://netflixtechblog.com